Loops, Prompt & Alert

for, while, do-while — Lab Worksheet

Name: _____ Section: _____ Date: _____

How to use this sheet: Try the loop examples in the console first — short answers only. Then move to the **Main Task**, where you'll build small interactive scripts using `prompt()`, `console.log()`, and `alert()` in an actual HTML page. **Part 3** then asks you to combine everything into two small programs.

Part 1 — Quick Checks: for, while, do-while

TRY THIS · CONSOLE

```
for (let i = 1; i <= 5; i++) { console.log(i); }
```

Q1. What printed? A `for` loop has three parts separated by semicolons — a **starting value**, a **condition**, and an **update step**. Based on what you see, what do you think each of the three parts is doing?

TRY THIS · CONSOLE

```
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
```

Q2. Did this print the same thing as the `for` loop? What do you have to remember to do manually with a `while` loop that the `for` loop handled for you automatically?

Q3. What do you think would happen if you forgot to include `i++` in this `while` loop?

TRY THIS · CONSOLE

```
let i = 10;
do {
  console.log(i);
  i++;
} while (i < 5);
```

Q4. The condition (`i < 5`) is false from the very start — but did the code run at least once anyway? How does a `do-while` loop differ from a `while` loop in *when* it checks its condition?

Q5. In your own words, when would you reach for a `for` loop, a `while` loop, or a `do-while` loop? Give one situation where `do-while`'s "runs at least once" behavior would actually matter.

Part 2 — Main Task: Prompt, Parsing & Alert

Build this in a new file, `loops-and-dialogs.html`, linked to `script.js`. Since `prompt()` and `alert()` are real browser popups, open the page in your actual browser tab — not just the console — to see them.

MAIN TASK · STEP 1 — PROMPT & CONSOLE.LOG BASICS

```
let name = prompt("What is your name?");
console.log(name);
```

MAIN TASK · STEP 2 — REWRITE WITH A TEMPLATE LITERAL

Replace Step 1's `console.log` with a template literal version:

```
console.log(`Hello, ${name}!`);
```

Concept — prompt() Always Returns a String. Even if the user types `21`, `prompt()` hands it back as the text `"21"`, not the number `21`. To use it in math, convert it first with `parseInt()`, `parseFloat()`, or `Number()`.

Function	What it does	Example
`parseInt(str)`	Reads whole-number digits from the start; stops at the first character that isn't part of that number, dropping any decimal part	`parseInt("42px")` → `42`
`parseFloat(str)`	Same as `parseInt()`, but keeps a decimal point if there is one	`parseFloat("3.14 kg")` → `3.14`
`Number(str)`	Converts the *entire* string at once; if any part of it isn't a valid number, the result is `NaN`	`Number("42")` → `42`, but `Number("42px")` → `NaN`

MAIN TASK · STEP 3 — CONVERTING PROMPT INPUT TO A NUMBER

```
let ageInput = prompt("How old are you?");
let age = parseInt(ageInput);
console.log(`Next year you'll be ${age + 1}.`);
```

Q6. If you had skipped `parseInt()` and used the raw `ageInput` string directly in `ageInput + 1`, what would print instead of the expected number — and why?

MAIN TASK · STEP 4 — DO-WHILE + PROMPT (EXPENSE CALCULATOR)

Use a `do-while` loop to keep asking for an expense amount, adding each one to a running total, and stopping once the user types "done":

```
let total = 0;
let amount;
do {
  amount = prompt("Enter an expense (or type 'done' to finish):");
  if (amount !== "done") {
    total += parseFloat(amount);
  }
} while (amount !== "done");
console.log(`Total expenses: ${total}`);
```

MAIN TASK · STEP 5 — FOR LOOP + ALERT

Use a `for` loop to show 3 reminder alerts in a row:

```
for (let i = 1; i <= 3; i++) {
  alert(`Reminder ${i} of 3: Don't forget to water the plants!`);
}
```

Q7. In your own words, what's the real difference between a `for` loop and a `while` loop? When would you reach for one instead of the other?

Q8. In Step 4, if you'd used a `while` loop instead of `do-while`, would the prompt still show at least once if the user somehow typed "done" right away? When will it help us that `do-while` guarantees at least one run?

Part 3 — Implementation Tasks: Loops, Prompt & Parsing

Time to combine everything from Parts 1 and 2 into two small programs of your own. Build each one step by step, checking your work in the browser as you go — don't wait until the end to test it.

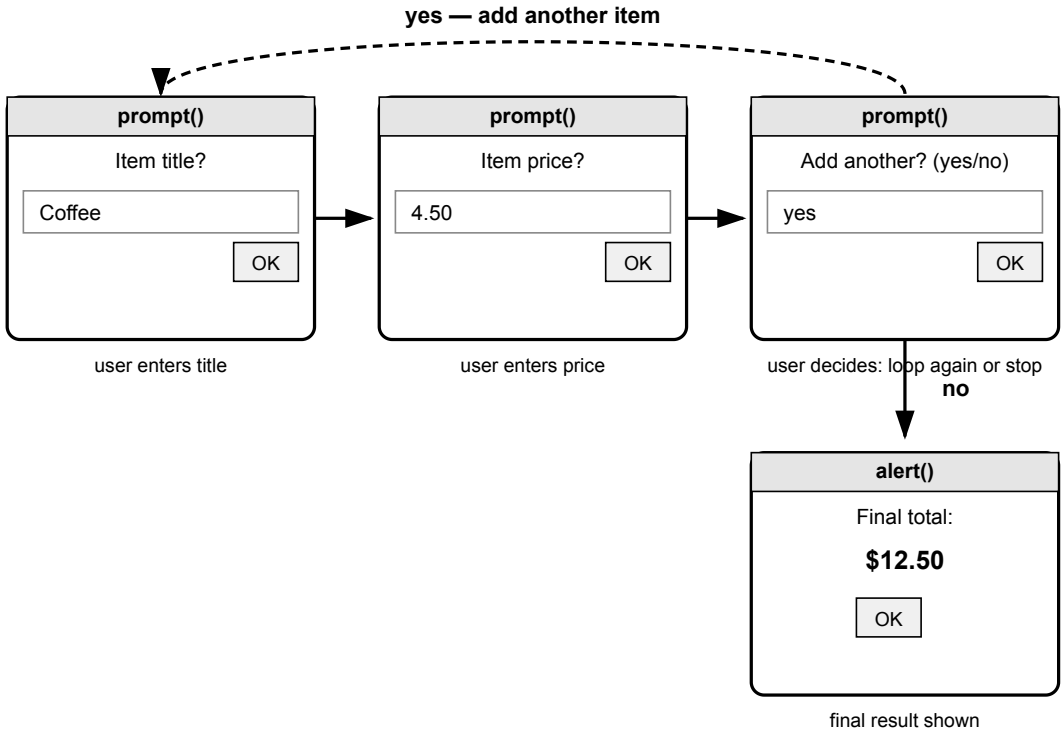
IMPLEMENTATION TASK 1 · SHOPPING CART TOTAL

Build a script that:

- Uses a `do-while` loop to repeatedly add items to a cart
- Each time through the loop, `prompt()`s for an item's **title**, then `prompt()`s for its **price** (convert it with `parseFloat()`)
- Uses `console.log()` to print that item's title and price
- Keeps a running **total** — add each item's price to it, and `console.log()` the updated total right alongside each item
- Asks "Add another item? (yes/no)" and stops the loop once the user types "no"
- After the loop ends, show the final total in an `alert()`, e.g. "Final total: 12.50"

TRY THIS · REFERENCE

The mockup below shows one possible run of the finished script — the sequence of popups a user would see, not how your code should be structured:



Q9. Where did you declare your running-total variable — inside the loop, or outside it? What would go wrong if you declared it inside?

Q10. Without pasting your actual code, write a short pseudocode of what your script does:

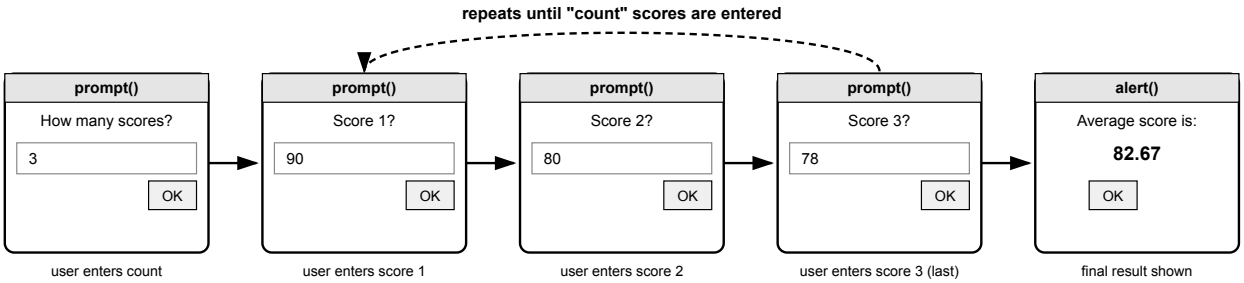
IMPLEMENTATION TASK 2 · AVERAGE SCORE CALCULATOR

Build a script that asks the user how many test scores they want to enter, then collects that many scores one at a time, and finally shows the class average in an `alert()`, e.g. “Your average score is: 84.5”.

You decide: which loop fits this best, how you’ll keep the running total, and which functions you’ll need to turn the prompted strings into numbers you can do math with.

TRY THIS · REFERENCE

The mockup below shows one possible run of the finished script — the sequence of popups a user would see, not how your code should be structured:



Q11. Which JavaScript functions did you use to convert the prompted strings into numbers? Did you pick a different one for the count than for each score?